GUJARAT TECHNOLOGICAL UNIVERSITY

Fundamentals of Software Development

Course Code: DI02016021 | Comprehensive Diploma Study Guide

Unit 1: Introduction to Software Development Process

1.1 Software & Its Characteristics

Definition: Software is a set of computer programs, procedures, and related documentation that tells a computer exactly how to operate and perform specific tasks.

3 Key Uses:

- Executes user commands accurately.
- Automates manual calculation processes.
- Stores and tracks digital information securely.

3 Core Features:

- Engineered instead of manufactured.
- Does not wear out physically over time.
- Highly customizable based on client needs.

Real-Life Application: A mobile banking application like GPay that processes payments by executing backend source code instructions safely.

1.2 Web-Based Applications

Definition: A web-based application is a software system that runs on a remote server and is accessed by users over the internet using a standard web browser interface.

3 Key Uses:

- Allows global product sales online.
- Enables remote working via cloud systems.
- Provides instant access to live databases.

3 Core Features:

- Network-intensive data transmission.
- Requires cross-browser layout compatibility.
- Continuous live security threat monitoring.

Real-Life Application: An online shopping website like Amazon where buyers browse catalogs and make purchases using any standard browser.

1.3 Software Engineering: A Layered Technology

Definition: Software engineering is an organized, disciplined approach to developing high-quality software using a structured infrastructure composed of tools, methods, processes, and a commitment to quality.

3 Key Uses:

- Ensures software products are reliable.
- Controls budget overheads during building.
- Maintains code standards across large teams.

3 Core Features:

- Quality Focus forms the bedrock layer.
- Process defines framework framework steps.
- Methods provide technical 'how-to' rules.

Real-Life Application: An IT company following strict coding guidelines, code review methods, and automated tracking tools to build corporate software.

1.4 Software Myths

Definition: Software myths are false beliefs or old assumptions held by managers, developers, or clients that cause project delays, poor quality, and misunderstandings.

3 Key Uses:

- Identifies bad developer habits early.
- Aligns realistic client expectations.
- Prevents management from rushing code work.

3 Core Features:

- Misleading short-term solutions.
- Widely believed by non-technical managers.
- Contradicts modern engineering facts.

Real-Life Application: A client believing that adding more programmers to a late project will speed it up, which actually delays it further due to training overhead.

Unit 2: Software Life Cycle Models

2.1 The Waterfall Model

Definition: The Waterfall model is a traditional, linear development model where progress flows strictly downwards through sequential phases like requirements, design, implementation, and testing.

3 Key Uses:

- Perfect for fixed, clear requirements.
- Easy to track and manage progress.
- Ensures thorough documentation at each step.

3 Core Features:

- Sequential step-by-step pipeline.
- No working software till late stages.
- Highly rigid with no backward loop changes.

Real-Life Application: Building a simple calculator system where all features are fully documented and locked before code writing begins.

2.2 The Prototyping Model

Definition: The Prototyping model builds a quick, working toy version of the software first to let clients test features and give feedback before real development starts.

3 Key Uses:

- Clarifies confusing user needs early.
- Reduces risk of building wrong products.
- Improves client satisfaction via early demos.

3 Core Features:

- Built rapidly with temporary code.
- Acts as an interactive requirement model.
- Requires active and constant customer reviews.

Real-Life Application: Creating a non-functional UI layout of a hospital portal on paper or wireframe tools to confirm button placements with doctors.

2.3 Agile Development

Definition: Agile development is an iterative approach that focuses on continuous feedback, rapid code delivery, flexible planning, and close teamwork.

3 Key Uses:

- Adapts smoothly to fast changing requirements.
- Delivers working code modules quickly.
- Maintains high customer involvement.

3 Core Features:

- Iterative and incremental work blocks.
- Daily team synchronization meetings.
- Focuses on running software over heavy documentation.

Real-Life Application: Developing a food delivery app by launching a basic login and map tracking system first, then adding promo codes next week.

Unit 3: Software Requirement Analysis

3.1 Requirement Gathering & Analysis

Definition: Requirement gathering and analysis is the initial phase where developers interview stakeholders to collect, organize, and understand what features the software must have.

3 Key Uses:

- Prevents missing critical system operations.
- Resolves arguments between different users.
- Sets clear limits on project delivery scope.

3 Core Features:

- Involves extensive stakeholder interaction.
- Discovers hidden business workflows.
- Eliminates vague, confusing statements.

Real-Life Application: Meeting with school principals and teachers to note down exactly how they want an automated student attendance system to send alerts.

3.2 Software Requirement Specification (SRS)

Definition: An SRS is an official document that describes the complete functional, technical, and operational requirements of a software project before building begins.

3 Key Uses:

- Acts as a legal agreement with clients.
- Provides a roadmap for code developers.
- Serves as the baseline map for testers.

3 Core Features:

- Clear, accurate, and completely verifiable.
- Independent of programming languages.
- Organized cleanly into functional categories.

Real-Life Application: A locked 50-page PDF document that clearly details how a bank portal must process cash inputs, handle errors, and log audit entries.

3.3 Functional vs. Non-Functional Requirements

Definition: Functional requirements describe what specific actions the system must perform, while non-functional requirements define how well the system must operate.

3 Key Uses:

- Ensures core user operations are coded.
- Guarantees system safety and fast speed.
- Creates a robust and user-friendly experience.

3 Core Features:

- Functional covers inputs and behavior.
- Non-functional covers speed and uptime.
- Both must be fully measurable and clear.

Real-Life Application: An app must send an OTP text message when a user logs in (Functional), and that OTP text must arrive within 5 seconds (Non-Functional).

Unit 4: Software Project Management

4.1 Project Manager Responsibilities

Definition: A Software Project Manager is the professional responsible for planning, executing, tracking, and closing software development projects on time and within budget.

3 Key Uses:

- Keeps teams organized and productive.
- Ensures milestone deliveries meet deadlines.
- Manages client communications smoothly.

3 Core Features:

- Strong tracking and risk management skills.
- Handles team building and task allocation.
- Acts as a bridge between clients and coders.

Real-Life Application: A manager using digital task boards to check daily coding progress and talk to clients about feature adjustments.

4.2 Project Scheduling (Gantt & PERT Charts)

Definition: Project scheduling is the process of breaking down a project into tasks, arranging them in order, assigning team members, and creating timelines.

3 Key Uses:

- Prevents missed task dependencies.
- Visually tracks project progress easily.
- Optimizes team workflow distribution.

3 Core Features:

- Uses Work Breakdown Structures (WBS).
- Identifies critical paths to avoid delays.
- Shows visual bar timelines for tasks.

Real-Life Application: An IT team using a Gantt chart to map out 2 weeks for database design, followed by 3 weeks for core feature coding.

4.3 Risk Management

Definition: Risk management is the systematic process of identifying, analyzing, and preparing plans to minimize the impact of potential problems in a project.

3 Key Uses:

- Prevents sudden project failures.
- Saves money by avoiding major bugs.
- Prepares alternative plans for sudden issues.

3 Core Features:

- Proactive risk spotting before coding.
- Assigns likelihood and impact levels.
- Creates clear mitigation action steps.

Real-Life Application: A manager hiring a backup developer early in case the main cloud engineer leaves the company unexpectedly mid-project.

Unit 5: Software Design

5.1 Cohesion and Coupling

Definition: Cohesion measures how focused a single code module is on a single task, while coupling measures how dependent different modules are on each other.

3 Key Uses:

- Makes code highly reusable.
- Allows easy bug fixing without breaks.
- Simplifies long-term system modifications.

3 Core Features:

- High Cohesion means focused modules.
- Low Coupling means highly independent units.
- Aims for cleaner, modular architecture.

Real-Life Application: A payment module that only handles card validation (High Cohesion) and does not break if the login module changes (Low Coupling).

5.2 Data Flow Diagrams (DFD)

Definition: A Data Flow Diagram is a graphical model that shows how data moves through an information system, detailing inputs, processing steps, and outputs.

3 Key Uses:

- Explains system logic to non-programmers.
- Maps out full data boundaries clearly.
- Helps structure database designs easily.

3 Core Features:

- Uses primitive circles, rectangles, and arrows.
- Structured into multi-level breakdowns.
- Shows data motion, not control timing.

Real-Life Application: A Level-1 DFD diagram showing how customer order details flow into a kitchen processing system and update an inventory store.

5.3 User Interface (UI) Design

Definition: User Interface design focuses on building the visual layouts, screens, buttons, and menus that humans interact with when using a software product.

3 Key Uses:

- Makes complex software easy to use.
- Reduces user errors during operations.
- Speeds up daily user data entry tasks.

3 Core Features:

- Clean, consistent visual appearance.
- Provides clear error messages and help.
- Responsive across phones and desktops.

Real-Life Application: Designing a simple login screen with big input boxes, a green submit button, and a clear 'Forgot Password' link.

Unit 6: Software Testing

6.1 Software Testing Fundamentals

Definition: Software testing is the process of executing a program with the explicit intent of finding bugs, errors, and verifying that it matches requirements.

3 Key Uses:

- Guarantees a bug-free user release.
- Saves massive costs of post-launch fixes.
- Ensures system safety and reliable code.

3 Core Features:

- Involves dynamic code execution paths.
- Requires well-planned test case inputs.
- Compares real outputs against expected maps.

Real-Life Application: Running a newly built login page with a wrong password to confirm that an error message pops up correctly.

6.2 Verification vs. Validation

Definition: Verification ensures the software is being built correctly according to specs, while Validation ensures the right product is being built to satisfy user needs.

3 Key Uses:

- Catches architecture flaws very early.
- Guarantees the client gets what they paid for.
- Maintains high quality during work stages.

3 Core Features:

- Verification reviews designs and documents.
- Validation runs and tests real software.
- Combines reviews with real execution proofs.

Real-Life Application: Reviewing the database architectural diagram (Verification) and checking if real cash dispenses from an ATM machine (Validation).

6.3 Black-Box vs. White-Box Testing

Definition: Black-box testing checks software functionality from the outside without knowing the inner code, while White-box testing inspects internal code structures and logic pathways directly.

3 Key Uses:

- Tests real-world end-user behaviors.
- Validates exact structural code paths.
- Ensures deep logical loops run safely.

3 Core Features:

- Black-box acts blind to internal code.
- White-box requires structural coding skill.
- Combines outer inputs with inner code checks.

Real-Life Application: Typing text into a signup form box to see if it works (Black-box) and reading loop criteria lines inside a Java file (White-box).